

PROJECT: BUILD A DATA MART IN SQL

```
mirror_mod = modifier_obj.  
Set mirror object to mirror.  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES ----  
  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

ENTITY RELATIONSHIP MODEL - OVERVIEW

- Conceptual model for the Airbnb use case (roles, listings, bookings, payments, reviews)
- Normalized structure with >20 entities and defined relationships
- Basis for the SQL implementation in PostgreSQL

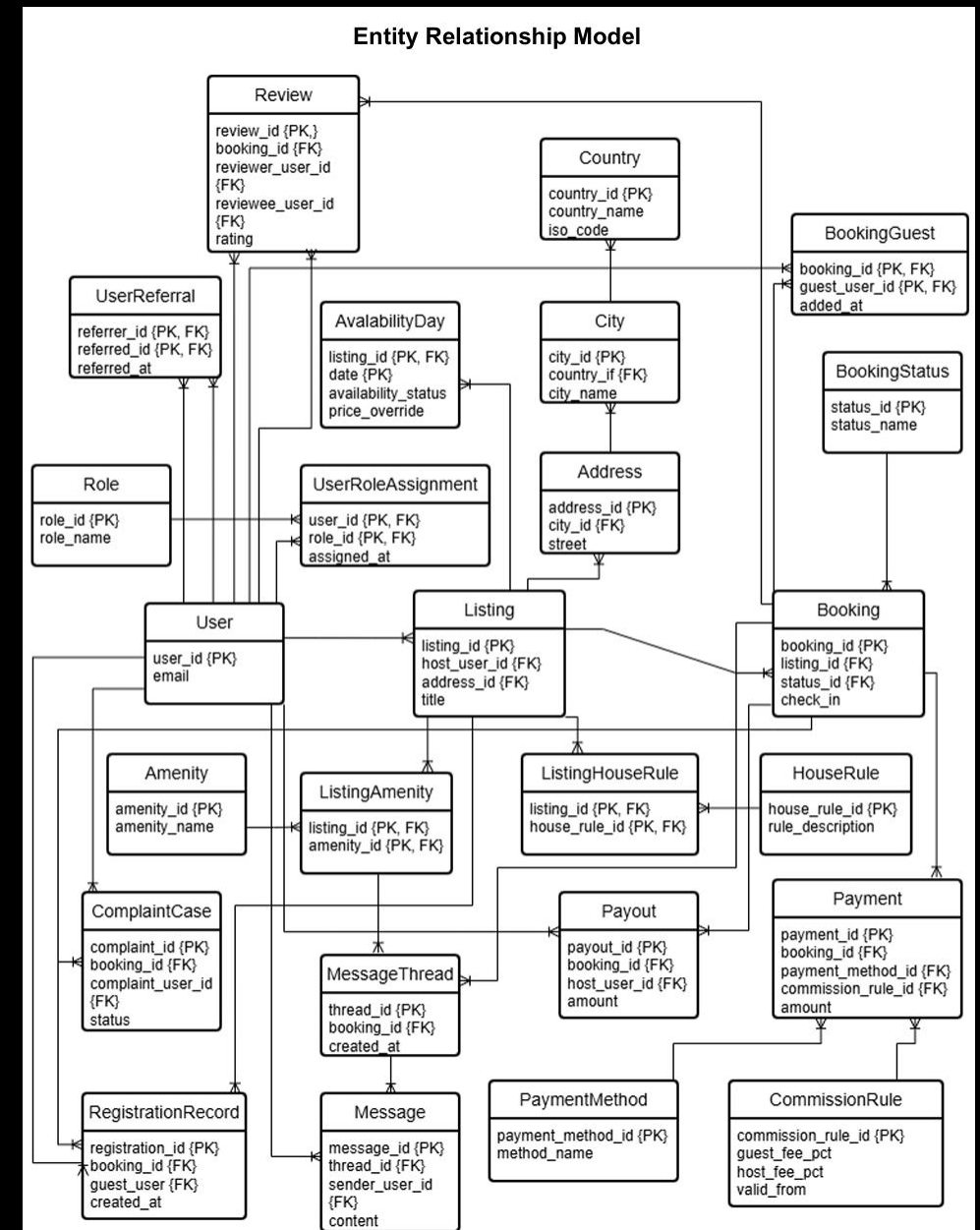


TABLE: ROLE

Stores all possible user roles within the platform (e.g. Guest, Host, Admin).

```
5 CREATE TABLE IF NOT EXISTS airbnb.role
6 (
7     role_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     role_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
9     CONSTRAINT role_pkey PRIMARY KEY (role_id),
10    CONSTRAINT role_role_name_key UNIQUE (role_name)
11 )
```

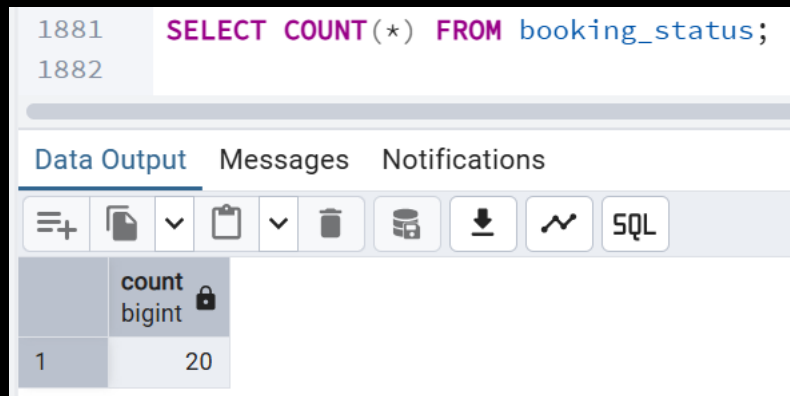
```
441 SELECT COUNT(*) FROM airbnb.role;
442 |
443
```

Data Output		Messages	Notifications
			Showing rows: 1 to 1
	count bigint		
1	20		

TABLE: BOOKING_STATUS

Stores all possible lifecycle states of a booking (e.g. requested, confirmed, completed, cancelled).

```
5 CREATE TABLE IF NOT EXISTS airbnb.booking_status
6 (
7     status_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     status_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
9     CONSTRAINT booking_status_pkey PRIMARY KEY (status_id),
10    CONSTRAINT booking_status_status_name_key UNIQUE (status_name)
11 )
```



The screenshot shows a database query interface. At the top, a SQL query is entered: `SELECT COUNT(*) FROM booking_status;`. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'count' (type 'bigint') and a single row with the value '20'.

	count 
1	20

The row count of the booking_status table was increased to 20 entries in order to comply with the „~20 rows per table“ guideline mentioned in the Phase 2 feedback.

TABLE: PAYMENT_METHOD

Stores all available payment options that guests can use to complete a booking transaction.

```
5 CREATE TABLE IF NOT EXISTS airbnb.payment_method
6 (
7     payment_method_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     method_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
9     CONSTRAINT payment_method_pkey PRIMARY KEY (payment_method_id),
10    CONSTRAINT payment_method_method_name_key UNIQUE (method_name)
11 )
```

```
445 SELECT COUNT(*) FROM airbnb.payment_method;
446
```

Data Output Messages Notifications



	count bigint
1	20

TABLE: COMMISSION_RULE

Defines the commission configuration used to calculate platform fees for guests and hosts over time.

```
5 CREATE TABLE IF NOT EXISTS airbnb.commission_rule
6 (
7     commission_rule_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     guest_fee_pct numeric(5,2) NOT NULL,
9     host_fee_pct numeric(5,2) NOT NULL,
10    valid_from date NOT NULL,
11    valid_to date,
12    CONSTRAINT commission_rule_pkey PRIMARY KEY (commission_rule_id),
13    CONSTRAINT chk_commission_guest_pct CHECK (guest_fee_pct >= 0::numeric AND guest_fee_pct <= 100::numeric),
14    CONSTRAINT chk_commission_host_pct CHECK (host_fee_pct >= 0::numeric AND host_fee_pct <= 100::numeric)
15 )
```

```
447 SELECT COUNT(*) FROM airbnb.commission_rule;
448
```

Data Output Messages Notifications

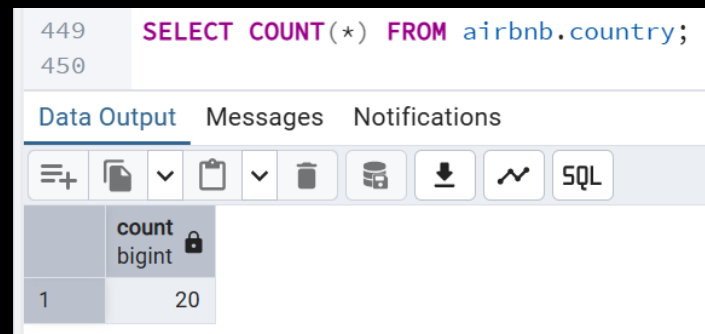


	count bigint
1	20

TABLE: COUNTRY

Stores all countries in which listings are located and serves as the top-level geographic reference.

```
5 CREATE TABLE IF NOT EXISTS airbnb.country
6 (
7     country_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     country_name character varying(100) COLLATE pg_catalog."default" NOT NULL,
9     iso_code character varying(3) COLLATE pg_catalog."default" NOT NULL,
10    CONSTRAINT country_pkey PRIMARY KEY (country_id),
11    CONSTRAINT country_iso_code_key UNIQUE (iso_code)
12 )
```



The screenshot shows a database interface with a SQL query editor at the top and a results pane below. The query is `SELECT COUNT(*) FROM airbnb.country;`. The results pane has tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a single row with a column named 'count' of type 'bigint' and a value of 20. The interface includes various icons for actions like copy, paste, and download.

	count bigint
1	20

TABLE: CITY

Stores cities and links them to their corresponding country to ensure geographic normalization.

```
5 CREATE TABLE IF NOT EXISTS airbnb.city
6 (
7     city_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     country_id integer NOT NULL,
9     city_name character varying(100) COLLATE pg_catalog."default" NOT NULL,
10    CONSTRAINT city_pkey PRIMARY KEY (city_id),
11    CONSTRAINT fk_city_country FOREIGN KEY (country_id)
12        REFERENCES airbnb.country (country_id) MATCH SIMPLE
13        ON UPDATE NO ACTION
14        ON DELETE NO ACTION
15 )
```

451	SELECT COUNT(*) FROM airbnb.city;
452	
Data Output Messages Notifications	
	count
	bigint
1	22

TABLE: ADDRESS

Stores the detailed address of each listing and connects it to the corresponding city.

```
5 CREATE TABLE IF NOT EXISTS airbnb.address
6 (
7     address_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     city_id integer NOT NULL,
9     street character varying(30) COLLATE pg_catalog."default" NOT NULL,
10    postal_code character varying(20) COLLATE pg_catalog."default" NOT NULL,
11    CONSTRAINT address_pkey PRIMARY KEY (address_id),
12    CONSTRAINT fk_address_city FOREIGN KEY (city_id)
13        REFERENCES airbnb.city (city_id) MATCH SIMPLE
14        ON UPDATE NO ACTION
15        ON DELETE NO ACTION
16 )
```

```
453 SELECT COUNT(*) FROM airbnb.address;
454
```

Data Output		Messages	Notifications
	count bigint		
1	66		

TABLE: APP_USER

Stores the master data of all platform users, including guests, hosts, and administrators.

```
5 CREATE TABLE IF NOT EXISTS airbnb.app_user
6 (
7     user_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     email character varying(250) COLLATE pg_catalog."default" NOT NULL,
9     phone character varying(50) COLLATE pg_catalog."default",
10    first_name character varying(100) COLLATE pg_catalog."default" NOT NULL,
11    last_name character varying(100) COLLATE pg_catalog."default" NOT NULL,
12    birth_date date,
13    created_at timestamp without time zone NOT NULL DEFAULT now(),
14    CONSTRAINT app_user_pkey PRIMARY KEY (user_id),
15    CONSTRAINT app_user_email_key UNIQUE (email)
16 )
```

```
455 SELECT COUNT(*) FROM airbnb.app_user;
456
```

Data Output		Messages	Notifications
         SQL			
	count  bigint		
1	20		

TABLE: LISTING

Stores all accommodations offered by hosts, including pricing and structural details.

```
5 CREATE TABLE IF NOT EXISTS airbnb.listing
6 (
7   listing_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8   host_user_id integer NOT NULL,
9   address_id integer NOT NULL,
10  title character varying(150) COLLATE pg_catalog."default" NOT NULL,
11  description character varying(500) COLLATE pg_catalog."default",
12  base_price numeric(10,2) NOT NULL,
13  max_guests integer NOT NULL,
14  created_at timestamp without time zone NOT NULL DEFAULT now(),
15  CONSTRAINT listing_pkey PRIMARY KEY (listing_id),
16  CONSTRAINT fk_listing_address FOREIGN KEY (address_id)
17    REFERENCES airbnb.address (address_id) MATCH SIMPLE
18    ON UPDATE NO ACTION
19    ON DELETE NO ACTION,
20  CONSTRAINT fk_listing_host FOREIGN KEY (host_user_id)
21    REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
22    ON UPDATE NO ACTION
23    ON DELETE NO ACTION,
24  CONSTRAINT chk_listing_base_price CHECK (base_price >= 0::numeric),
25  CONSTRAINT chk_listing_max_guests CHECK (max_guests >= 0)
26 )
```

458	SELECT COUNT(*) FROM airbnb.listing;
459	
Data Output Messages Notifications	
	
	count 
1	20

TABLE: BOOKING

Stores all booking transactions, including booking period, status, and total amount.

```
5 CREATE TABLE IF NOT EXISTS airbnb.booking
6 (
7   booking_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8   listing_id integer NOT NULL,
9   status_id integer NOT NULL,
10  check_in date NOT NULL,
11  check_out date NOT NULL,
12  created_at timestamp without time zone NOT NULL DEFAULT now(),
13  CONSTRAINT booking_pkey PRIMARY KEY (booking_id),
14  CONSTRAINT fk_booking_listing FOREIGN KEY (listing_id)
15    REFERENCES airbnb.listing (listing_id) MATCH SIMPLE
16    ON UPDATE NO ACTION
17    ON DELETE NO ACTION,
18  CONSTRAINT fk_booking_status FOREIGN KEY (status_id)
19    REFERENCES airbnb.booking_status (status_id) MATCH SIMPLE
20    ON UPDATE NO ACTION
21    ON DELETE NO ACTION,
22  CONSTRAINT chk_booking_dates CHECK (check_out > check_in)
23 )
```

460	SELECT COUNT(*) FROM airbnb.booking;
461	
Data Output Messages Notifications	
	count bigint
1	20

TABLE: USER_ROLE_ASSIGNMENT

Stores one or multiple roles to users and implements the many-to-many relationship between users and roles.

```
5 CREATE TABLE IF NOT EXISTS airbnb.user_role_assignment
6 (
7     user_id integer NOT NULL,
8     role_id integer NOT NULL,
9     assigned_at timestamp without time zone NOT NULL DEFAULT now(),
10    CONSTRAINT pk_user_role_assignment PRIMARY KEY (user_id, role_id),
11    CONSTRAINT fk_ura_role FOREIGN KEY (role_id)
12        REFERENCES airbnb.role (role_id) MATCH SIMPLE
13        ON UPDATE NO ACTION
14        ON DELETE NO ACTION,
15    CONSTRAINT fk_ura_user FOREIGN KEY (user_id)
16        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
17        ON UPDATE NO ACTION
18        ON DELETE NO ACTION
19 )
```

```
462 SELECT COUNT(*) FROM airbnb.user_role_assignment;
463
```

Data Output		Messages	Notifications
	count bigint		
1	31		

TABLE: BOOKING_GUEST

Links bookings to guests and represents the relationship between users and booking transactions.

```
5 CREATE TABLE IF NOT EXISTS airbnb.booking_guest
6 (
7     booking_id integer NOT NULL,
8     guest_user_id integer NOT NULL,
9     added_at timestamp without time zone NOT NULL DEFAULT now(),
10    CONSTRAINT pk_booking_guest PRIMARY KEY (booking_id, guest_user_id),
11    CONSTRAINT fk_bg_booking FOREIGN KEY (booking_id)
12        REFERENCES airbnb.booking (booking_id) MATCH SIMPLE
13        ON UPDATE NO ACTION
14        ON DELETE NO ACTION,
15    CONSTRAINT fk_bg_user FOREIGN KEY (guest_user_id)
16        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
17        ON UPDATE NO ACTION
18        ON DELETE NO ACTION
19 )
```

464	SELECT COUNT(*) FROM airbnb.booking_guest;
465	
Data Output Messages Notifications	
	count
	bigint
1	20

TABLE: AMENITY

Stores all available amenities that can be assigned to listings (e.g. Wi-Fi, parking, kitchen).

```
5 CREATE TABLE IF NOT EXISTS airbnb.amenity
6 (
7     amenity_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     amenity_name character varying(250) COLLATE pg_catalog."default" NOT NULL,
9     CONSTRAINT amenity_pkey PRIMARY KEY (amenity_id),
10    CONSTRAINT amenity_amenity_name_key UNIQUE (amenity_name)
11 )
```

```
466 SELECT COUNT(*) FROM airbnb.amenity;
467
```

Data Output		Messages	Notifications
	count		
	bigint		
1	20		

TABLE: LISTING_AMENITY

Implements the many-to-many relationship between listings and amenities.

```
5 CREATE TABLE IF NOT EXISTS airbnb.listing_amenity
6 (
7     listing_id integer NOT NULL,
8     amenity_id integer NOT NULL,
9     CONSTRAINT pk_listing_amenity PRIMARY KEY (listing_id, amenity_id),
10    CONSTRAINT fk_la_amenity FOREIGN KEY (amenity_id)
11        REFERENCES airbnb.amenity (amenity_id) MATCH SIMPLE
12        ON UPDATE NO ACTION
13        ON DELETE NO ACTION,
14    CONSTRAINT fk_la_listing FOREIGN KEY (listing_id)
15        REFERENCES airbnb.listing (listing_id) MATCH SIMPLE
16        ON UPDATE NO ACTION
17        ON DELETE NO ACTION
18 )
```

468	SELECT COUNT(*) FROM airbnb.listing_amenity;
469	
Data Output Messages Notifications	
	count
	bigint
1	60

TABLE: HOUSE_RULE

Stores all house rules that define behavioral requirements for guests within a listing.

```
5 CREATE TABLE IF NOT EXISTS airbnb.house_rule
6 (
7     house_rule_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     rule_description character varying(500) COLLATE pg_catalog."default" NOT NULL,
9     CONSTRAINT house_rule_pkey PRIMARY KEY (house_rule_id),
10    CONSTRAINT house_rule_rule_description_key UNIQUE (rule_description)
11 )
```

```
470 SELECT COUNT(*) FROM airbnb.house_rule;
471
```

Data Output		Messages	Notifications
	count		
	bigint		
1	20		

TABLE: LISTING_HOUSE_RULE

Implements the many-to-many relationship between listings and house rules.

```
5 CREATE TABLE IF NOT EXISTS airbnb.listing_house_rule
6 (
7     listing_id integer NOT NULL,
8     house_rule_id integer NOT NULL,
9     CONSTRAINT pk_listing_house_rule PRIMARY KEY (listing_id, house_rule_id),
10    CONSTRAINT fk_lhr_house_rule FOREIGN KEY (house_rule_id)
11        REFERENCES airbnb.house_rule (house_rule_id) MATCH SIMPLE
12        ON UPDATE NO ACTION
13        ON DELETE NO ACTION,
14    CONSTRAINT fk_lhr_listing FOREIGN KEY (listing_id)
15        REFERENCES airbnb.listing (listing_id) MATCH SIMPLE
16        ON UPDATE NO ACTION
17        ON DELETE NO ACTION
18 )
```

```
472 SELECT COUNT(*) FROM airbnb.listing_house_rule;
473
```

Data Output		Messages	Notifications
	count bigint		
1	40		

TABLE: AVAILABILITY_DAY

Stores the day-level availability calendar for each listing, including optional price overrides and blocked days.

```
5 CREATE TABLE IF NOT EXISTS airbnb.availability_day
6 (
7     listing_id integer NOT NULL,
8     day_date date NOT NULL,
9     availability_status character varying(20) COLLATE pg_catalog."default" NOT NULL,
10    price_override numeric(10,2),
11    CONSTRAINT pk_availability_day PRIMARY KEY (listing_id, day_date),
12    CONSTRAINT fk_avd_listing FOREIGN KEY (listing_id)
13        REFERENCES airbnb.listing (listing_id) MATCH SIMPLE
14        ON UPDATE NO ACTION
15        ON DELETE NO ACTION,
16    CONSTRAINT chk_avd_price CHECK (price_override IS NULL OR price_override >= 0::numeric)
17 )
```

474	SELECT COUNT(*) FROM airbnb.availability_day;
475	
Data Output Messages Notifications	
	count
1	600

TABLE: MESSAGE_THREAD

Represents structured communication channels between guests and hosts regarding bookings.

```
5 CREATE TABLE IF NOT EXISTS airbnb.message_thread
6 (
7   thread_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8   booking_id integer NOT NULL,
9   created_at timestamp without time zone NOT NULL DEFAULT now(),
10  CONSTRAINT message_thread_pkey PRIMARY KEY (thread_id),
11  CONSTRAINT fk_mt_booking FOREIGN KEY (booking_id)
12    REFERENCES airbnb.booking (booking_id) MATCH SIMPLE
13    ON UPDATE NO ACTION
14    ON DELETE NO ACTION
15 )
```

```
476 SELECT COUNT(*) FROM airbnb.message_thread;
477
```

Data Output		Messages	Notifications
	count		
	bigint		
1	20		

TABLE: MESSAGE

Stores individual messages exchanged within a message thread between guests and hosts.

```
5 CREATE TABLE IF NOT EXISTS airbnb.message
6 (
7     message_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     thread_id integer NOT NULL,
9     sender_user_id integer NOT NULL,
10    send_at timestamp without time zone NOT NULL DEFAULT now(),
11    content character varying(1000) COLLATE pg_catalog."default" NOT NULL,
12    CONSTRAINT message_pkey PRIMARY KEY (message_id),
13    CONSTRAINT fk_msg_thread FOREIGN KEY (thread_id)
14        REFERENCES airbnb.message_thread (thread_id) MATCH SIMPLE
15        ON UPDATE NO ACTION
16        ON DELETE NO ACTION,
17    CONSTRAINT fk_msg_user FOREIGN KEY (sender_user_id)
18        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
19        ON UPDATE NO ACTION
20        ON DELETE NO ACTION
21 )
```

478	SELECT COUNT(*) FROM airbnb.message;
479	
Data Output Messages Notifications	
         SQL	
	count bigint 
1	60

TABLE: COMPLAINT_CASE

Stores complaint records related to bookings, allowing administrators to review and resolve disputes.

```
5 CREATE TABLE IF NOT EXISTS airbnb.complaint_case
6 (
7     complaint_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     booking_id integer NOT NULL,
9     complainant_user_id integer NOT NULL,
10    category character varying(80) COLLATE pg_catalog."default" NOT NULL,
11    status character varying(30) COLLATE pg_catalog."default" NOT NULL,
12    opened_at timestamp without time zone NOT NULL DEFAULT now(),
13    CONSTRAINT complaint_case_pkey PRIMARY KEY (complaint_id),
14    CONSTRAINT fk_cc_booking FOREIGN KEY (booking_id)
15        REFERENCES airbnb.booking (booking_id) MATCH SIMPLE
16        ON UPDATE NO ACTION
17        ON DELETE NO ACTION,
18    CONSTRAINT fk_cc_user FOREIGN KEY (complainant_user_id)
19        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
20        ON UPDATE NO ACTION
21        ON DELETE NO ACTION
22 )
```

482	SELECT COUNT(*) FROM airbnb.complaint_case;
483	
Data Output Messages Notifications	
	count bigint
1	20

TABLE: PAYMENT

Stores all incoming payments from guests for bookings, including payment status and applied commission rule.

```
5 CREATE TABLE IF NOT EXISTS airbnb.payment
6 (
7     payment_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     booking_id integer NOT NULL,
9     payment_method_id integer NOT NULL,
10    commission_rule_id integer NOT NULL,
11    amount numeric(10,2) NOT NULL,
12    paid_at timestamp without time zone NOT NULL DEFAULT now(),
13    payment_status character varying(30) COLLATE pg_catalog."default" NOT NULL,
14    CONSTRAINT payment_pkey PRIMARY KEY (payment_id),
15    CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id)
16        REFERENCES airbnb.booking (booking_id) MATCH SIMPLE
17        ON UPDATE NO ACTION
18        ON DELETE NO ACTION,
19    CONSTRAINT fk_payment_commission FOREIGN KEY (commission_rule_id)
20        REFERENCES airbnb.commission_rule (commission_rule_id) MATCH SIMPLE
21        ON UPDATE NO ACTION
22        ON DELETE NO ACTION,
23    CONSTRAINT fk_payment_method FOREIGN KEY (payment_method_id)
24        REFERENCES airbnb.payment_method (payment_method_id) MATCH SIMPLE
25        ON UPDATE NO ACTION
26        ON DELETE NO ACTION,
27    CONSTRAINT chk_payment_amount CHECK (amount >= 0::numeric)
28 )
```

484	SELECT COUNT(*) FROM airbnb.payment;
485	
Data Output Messages Notifications	
	count bigint
1	20

TABLE: PAYOUT

Stores outgoing payments from the platform to hosts after successful booking completion.

```
5 CREATE TABLE IF NOT EXISTS airbnb.payout
6 (
7     payout_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     booking_id integer NOT NULL,
9     host_user_id integer NOT NULL,
10    amount numeric(10,1) NOT NULL,
11    payout_at timestamp without time zone,
12    payout_status character varying(30) COLLATE pg_catalog."default" NOT NULL,
13    CONSTRAINT payout_pkey PRIMARY KEY (payout_id),
14    CONSTRAINT fk_payout_booking FOREIGN KEY (booking_id)
15        REFERENCES airbnb.booking (booking_id) MATCH SIMPLE
16        ON UPDATE NO ACTION
17        ON DELETE NO ACTION,
18    CONSTRAINT fk_payout_user FOREIGN KEY (host_user_id)
19        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
20        ON UPDATE NO ACTION
21        ON DELETE NO ACTION,
22    CONSTRAINT chk_payout_amount CHECK (amount >= 0::numeric)
23 )
```

486	SELECT COUNT(*) FROM airbnb.payout;
487	
Data Output Messages Notifications	
≡+ ▼ ▼ SQL	
	count
1	20

TABLE: REGISTRATION_RECORD

Stores legally required guest registration data for retention purposes (historicized records).

```
5 CREATE TABLE IF NOT EXISTS airbnb.registration_record
6 (
7     registration_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     booking_id integer NOT NULL,
9     guest_user_id integer NOT NULL,
10    created_at timestamp without time zone NOT NULL DEFAULT now(),
11    stored_until date NOT NULL,
12    CONSTRAINT registration_record_pkey PRIMARY KEY (registration_id),
13    CONSTRAINT fk_reg_booking FOREIGN KEY (booking_id)
14        REFERENCES airbnb.booking (booking_id) MATCH SIMPLE
15        ON UPDATE NO ACTION
16        ON DELETE NO ACTION,
17    CONSTRAINT fk_reg_user FOREIGN KEY (guest_user_id)
18        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
19        ON UPDATE NO ACTION
20        ON DELETE NO ACTION
21 )
```

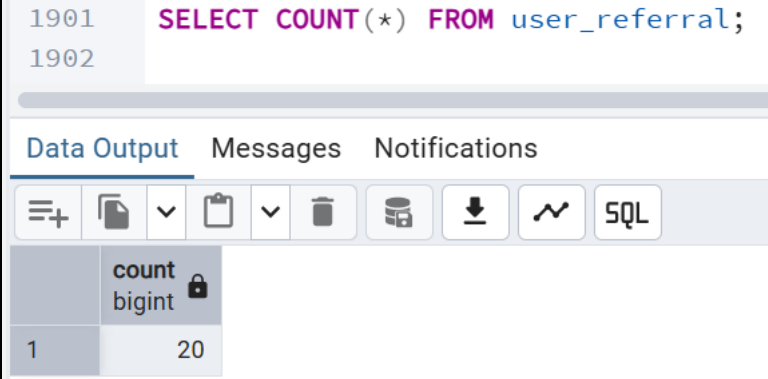
```
488 SELECT COUNT(*) FROM airbnb.registration_record;
489
```

Data Output		Messages	Notifications
	count bigint		
1	20		

TABLE: USER_REFERRAL

Represents the self-referencing relationship between users within the referral system.

```
5 CREATE TABLE IF NOT EXISTS airbnb.user_referral
6 (
7     referrer_user_id integer NOT NULL,
8     referred_user_id integer NOT NULL,
9     referred_at timestamp without time zone NOT NULL DEFAULT now(),
10    CONSTRAINT pk_user_referral PRIMARY KEY (referrer_user_id, referred_user_id),
11    CONSTRAINT fk_referred FOREIGN KEY (referred_user_id)
12        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
13        ON UPDATE NO ACTION
14        ON DELETE NO ACTION,
15    CONSTRAINT fk_referrer FOREIGN KEY (referrer_user_id)
16        REFERENCES airbnb.app_user (user_id) MATCH SIMPLE
17        ON UPDATE NO ACTION
18        ON DELETE NO ACTION
19 )
```



The screenshot shows a database interface with a SQL query editor at the top and a results pane below. The query is `SELECT COUNT(*) FROM user_referral;`. The results pane has tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with one row and two columns: 'count' and 'bigint'. The value in the 'count' column is 20.

	count
1	20

The row count of the user_referral table was increased to 20 entries in order to comply with the „~20 rows per table“ guideline mentioned in the Phase 2 feedback.

ANALYTICAL TEST CASES



VALIDATION OF
RELATIONAL INTEGRITY



DEMONSTRATION OF
BUSINESS LOGIC



AGGREGATION AND
REVENUE ANALYSIS

TEST CASE 1: BOOKING OVERVIEW

Deomonstrates the end-to-end booking process by joining booking, guest, host, listing, booking status, and payment information.

```
492  -- =====
493  -- TEST CASES
494  -- =====
495
496  SELECT
497      b.booking_id,
498      gu.first_name || ' ' || gu.last_name AS guest,
499      hu.first_name || ' ' || hu.last_name AS host,
500      l.title AS listing,
501      bs.status_name,
502      p.amount,
503      p.payment_status
504  FROM airbnb.booking b
505  JOIN airbnb.booking_guest bg ON bg.booking_id = b.booking_id
506  JOIN airbnb.app_user gu ON gu.user_id = bg.guest_user_id
507  JOIN airbnb.listing l ON l.listing_id = b.listing_id
508  JOIN airbnb.app_user hu ON hu.user_id = l.host_user_id
509  JOIN airbnb.booking_status bs ON bs.status_id = b.status_id
510  JOIN airbnb.payment p ON p.payment_id = b.payment_id
511  ORDER BY b.booking_id;
```

	booking_id integer	guest text	host text	listing character varying (150)	status_name character varying (50)	amount numeric (10,2)	payment_status character varying (30)
1	1	First11 Last...	First11 Last1	Listing1	confirmed	171.00	refunded
2	2	First12 Last...	First12 Last2	Listing2	confirmed	192.00	pending
3	3	First13 Last...	First13 Last3	Listing3	confirmed	213.00	paid
4	4	First14 Last...	First14 Last4	Listing4	confirmed	234.00	refunded
5	5	First15 Last...	First15 Last5	Listing5	confirmed	255.00	pending
6	6	First16 Last...	First16 Last6	Listing6	confirmed	276.00	paid
7	7	First17 Last...	First17 Last7	Listing7	confirmed	297.00	refunded
8	8	First18 Last...	First18 Last8	Listing8	confirmed	318.00	pending
9	9	First19 Last...	First19 Last9	Listing9	confirmed	339.00	paid
10	10	First20 Last...	First20 Last...	Listing10	confirmed	360.00	refunded
11	11	First11 Last...	First11 Last1	Listing11	confirmed	381.00	pending
12	12	First12 Last...	First12 Last2	Listing12	confirmed	402.00	paid
13	13	First13 Last...	First13 Last3	Listing13	confirmed	423.00	refunded
14	14	First14 Last...	First14 Last4	Listing14	confirmed	444.00	pending
15	15	First15 Last...	First15 Last5	Listing15	confirmed	465.00	paid
16	16	First16 Last...	First16 Last6	Listing16	confirmed	486.00	refunded
17	17	First17 Last...	First17 Last7	Listing17	confirmed	507.00	pending
18	18	First18 Last...	First18 Last8	Listing18	confirmed	528.00	paid
19	19	First19 Last...	First19 Last9	Listing19	confirmed	549.00	refunded
20	20	First20 Last...	First20 Last...	Listing20	confirmed	570.00	pending

TEST CASE 2: AVERAGE RATING PER HOST

Calculates the average host rating based on received reviews and demonstrates analytical aggregation with GROUP BY.

```
513 SELECT
514     hu.user_id,
515     hu.first_name || ' ' || hu.last_name AS host,
516     ROUND(AVG(r.rating), 2) AS avg_rating,
517     COUNT(r.review_id) AS total_reviews
518 FROM   airbnb.review r
519 JOIN   airbnb.app_user hu ON hu.user_id = r.reviewee_user_id
520 JOIN   airbnb.listing l ON l.host_user_id = hu.user_id
521 GROUP BY hu.user_id, hu.first_name, hu.last_name
522 ORDER BY avg_rating DESC;
```

Data Output					Messages	Notifications
	user_id [PK] integer	host text	avg_rating numeric	total_reviews bigint		
1	7	First7 Last7	5.00	4		
2	3	First3 Last3	5.00	4		
3	1	First1 Last1	5.00	4		
4	5	First5 Last5	5.00	4		
5	9	First9 Last9	5.00	4		
6	8	First8 Last8	4.00	4		
7	10	First10 Last...	4.00	4		
8	6	First6 Last6	4.00	4		
9	2	First2 Last2	4.00	4		
10	4	First4 Last4	4.00	4		

TEST CASE 3: REVENUE PER LISTING

Calculates the total generated revenue per listing and demonstrates financial aggregation within the data mart.

```
524 SELECT
525     l.listing_id,
526     l.title,
527     SUM(p.amount) AS total_revenue
528 FROM   airbnb.payment p
529 JOIN   airbnb.booking b ON b.booking_id = p.booking_id
530 JOIN   airbnb.listing l ON l.listing_id = b.listing_id
531 GROUP BY l.listing_id, l.title
532 ORDER BY total_revenue DESC;
```

	listing_id [PK] integer	title character varying (150)	total_revenue numeric
1	20	Listing20	570.00
2	19	Listing19	549.00
3	18	Listing18	528.00
4	17	Listing17	507.00
5	16	Listing16	486.00
6	15	Listing15	465.00
7	14	Listing14	444.00
8	13	Listing13	423.00
9	12	Listing12	402.00
10	11	Listing11	381.00
11	10	Listing10	360.00
12	9	Listing9	339.00
13	8	Listing8	318.00
14	7	Listing7	297.00
15	6	Listing6	276.00
16	5	Listing5	255.00
17	4	Listing4	234.00
18	3	Listing3	213.00
19	2	Listing2	192.00
20	1	Listing1	171.00

IMPLEMENTATION SUMMARY

- Implementation of a fully normalized relational data mart in PostgreSQL
- More than 20 entities with defined primary and foreign key relationships
- Enforcement of data integrity through constraints and checks
- All tables populated with test data for validation purposes
- Analytical test cases successfully demonstrate booking, rating, and revenue logic
- Final schema exported directly from PostgreSQL to ensure completeness and reproducibility